

PROJECT REPORT

Power Plant Energy Output Prediction

Regression Using an Artificial Neural Network (PyTorch)

Prepared by

Sankhasubhra Basu

1. Introduction

1.1 Project Overview

This project builds an Artificial Neural Network (ANN) using PyTorch to predict the net hourly electrical energy output of a Combined Cycle Power Plant, based on four ambient operating conditions recorded at the plant. Unlike the traditional machine learning models used in earlier projects (Linear Regression, Random Forest, etc.), this project uses a custom-built, multi-layer neural network trained through gradient descent and backpropagation — the foundation of deep learning.

The goal is to demonstrate a complete deep learning regression workflow: preparing tabular data for a neural network, defining a network architecture, training it with a proper training/validation loop, saving the best-performing version of the model, and evaluating its final accuracy.

1.2 Objectives

- Load and prepare the power plant sensor dataset for use with a neural network.
- Convert the data into PyTorch tensors and organise it into batches using DataLoader.
- Design a simple feed-forward ANN architecture suited to a regression problem.
- Train the network over multiple epochs, tracking both training loss and validation loss.
- Automatically save the best-performing version of the model during training.
- Evaluate the final model on unseen test data using MSE and R^2 score.

1.3 Tools and Libraries Used

The project was implemented in Python, using the following libraries:

- **Pandas and NumPy** — data loading and manipulation.
- **Matplotlib and Seaborn** — visualising the training and validation loss curves.
- **scikit-learn (sklearn)** — train-test splitting, feature scaling, and the R^2 evaluation metric.
- **PyTorch (torch, torch.nn, torch.optim)** — building, training, and evaluating the neural network.

2. Dataset Description

The dataset used is `powerplant_data.csv`, containing 9,568 hourly records collected from a Combined Cycle Power Plant, with 5 columns describing the ambient operating conditions and the resulting energy output.

Column Name	Full Meaning	Description
AT	Ambient Temperature	The surrounding air temperature at the plant
V	Exhaust Vacuum	The vacuum pressure in the plant's exhaust system
AP	Ambient Pressure	The surrounding atmospheric pressure
RH	Relative Humidity	The surrounding air humidity
PE	Net Energy Output	The net hourly electrical energy generated by the plant — the target variable

The four input features (AT, V, AP, RH) are all continuous ambient sensor readings that influence how efficiently the plant generates electricity, while PE (Net Energy Output) is the continuous value the neural network is trained to predict — making this a regression problem.

3. Loading the Dataset and Splitting It

Before building the neural network, the raw data was checked and prepared:

- Missing value check:** `df.isnull().sum()` confirmed there were zero missing values across all 5 columns, so no imputation was required.
- Feature-target split:** The four sensor readings (AT, V, AP, RH) were separated into the feature set X, while the energy output column (PE) was set as the target y.
- Train-test split:** The dataset was split into training and testing sets using an 80:20 ratio with a fixed random state, so the split is reproducible.
- Feature scaling:** All input features were standardised using `StandardScaler`, fit only on the training data and then applied to both the training and test sets. Scaling is especially important for neural networks, since features on very different numeric scales (e.g., AP in the thousands vs. RH in tens) can make training slower and less stable, as the network's weights would need to compensate for those scale differences during gradient descent.

4. Converting the Data into Tensors

PyTorch neural networks operate on tensors rather than plain NumPy arrays or Pandas DataFrames, so the scaled feature sets and target values were converted into PyTorch tensors of type `float32`. The target tensors were also reshaped from a flat list into a column format (using `.view(-1, 1)`), since PyTorch expects the target to have a shape that matches the network's output layer, which produces one predicted value per row.

A device check was also performed at the start of the notebook (`torch.device("cuda" if torch.cuda.is_available() else "cpu")`), so that training could automatically make use of a GPU if one was available, significantly speeding up the training process.

5. Creating the TensorDataset and DataLoader

The feature and target tensors were paired together using **PyTorch's TensorDataset**, and then wrapped in a **DataLoader** for both the training and test sets. The DataLoader automatically splits the data into smaller batches (a batch size of 32 rows was used here) and, for the training data, shuffles the rows before every epoch.

Training in small batches rather than on the entire dataset at once is a standard deep learning practice — it allows the model's weights to be updated more frequently within a single epoch and makes training more memory-efficient, while shuffling helps prevent the model from learning any unintended order-based patterns in the data.

6. Defining the ANN Architecture

A custom neural network class, ANN, was defined using PyTorch's `nn.Module`, with the following layer structure:

Layer	Type	Details
Input	—	4 input features (AT, V, AP, RH)
Hidden Layer 1	Linear + ReLU	4 inputs → 6 neurons, followed by a ReLU activation
Hidden Layer 2	Linear + ReLU	6 inputs → 6 neurons, followed by a ReLU activation
Output Layer	Linear	6 inputs → 1 output neuron (the predicted PE value)

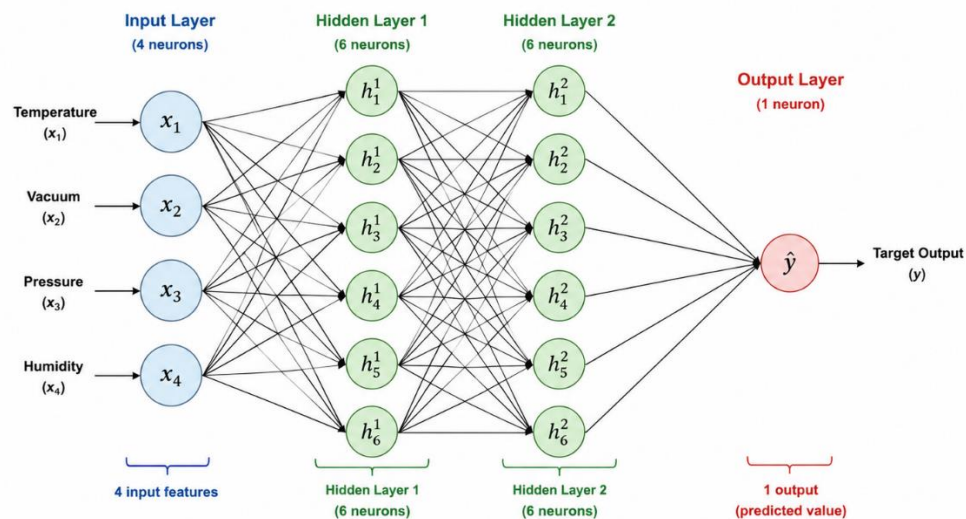


Figure 1: Architecture of the ANN used for prediction.

Two hidden layers were used, each followed by a **ReLU (Rectified Linear Unit) activation function**. ReLU was chosen because it introduces non-linearity into the network — without it, stacking multiple linear layers would mathematically collapse into behaving like a single linear layer, defeating the purpose of having a deep network. ReLU is also computationally efficient and helps avoid some of the training difficulties (such as vanishing gradients) that other activation functions can cause. The final output layer has a single neuron with no activation function, since the network needs to output a raw, continuous number (the predicted energy output) rather than a bounded or categorical value.

6.1 Loss Function and Optimizer

Mean Squared Error (MSELoss) was used as the loss function, which is the standard choice for regression tasks — it measures the average squared difference between the predicted and actual energy output, penalising larger errors more heavily. The **Adam optimizer** was used to update the network's weights during training, since it adapts the learning rate automatically for each parameter and generally converges faster and more reliably than plain gradient descent, making it a strong default choice for neural network training.

7. Training the ANN and Saving the Best Model

The model was trained for 50 epochs. In each epoch, the network went through the following process:

1. **Training phase:** For every batch from the training DataLoader, the model performed a forward pass (predicting outputs from the input batch), calculated the loss between the predictions and actual values, performed a backward pass to compute gradients, and then updated the network's weights using the optimizer. The average training loss across all batches was recorded for each epoch.
2. **Validation phase:** After each epoch's training was complete, the model was switched to evaluation mode and run on the test DataLoader with gradient computation disabled (`torch.no_grad()`), to measure how well it generalises to unseen data without allowing the test data to influence the model's weights.
3. **Saving the best model:** After every epoch, if the validation loss for that epoch was lower than the best validation loss seen so far, the model's parameters were saved to disk (`best_model.pt`). This ensures that, even if the model's performance were to fluctuate or overfit in later epochs, the version of the model with the strongest validation performance is always preserved.

The training loss and validation loss both started very high (around 2,05,000 and 2,02,000 respectively at epoch 1) and dropped sharply within the first 10–15 epochs, before levelling off and stabilising around epoch 35–40, ending at a **training loss of approximately 21.67** and a **validation loss of approximately 20.39** by epoch 50. Since the validation loss kept improving right up to the final epoch, the last saved checkpoint corresponds to the model trained for the full 50 epochs.

8. Plotting the Losses

The training and validation losses recorded across all 50 epochs were plotted together to visually inspect how the model learned over time.

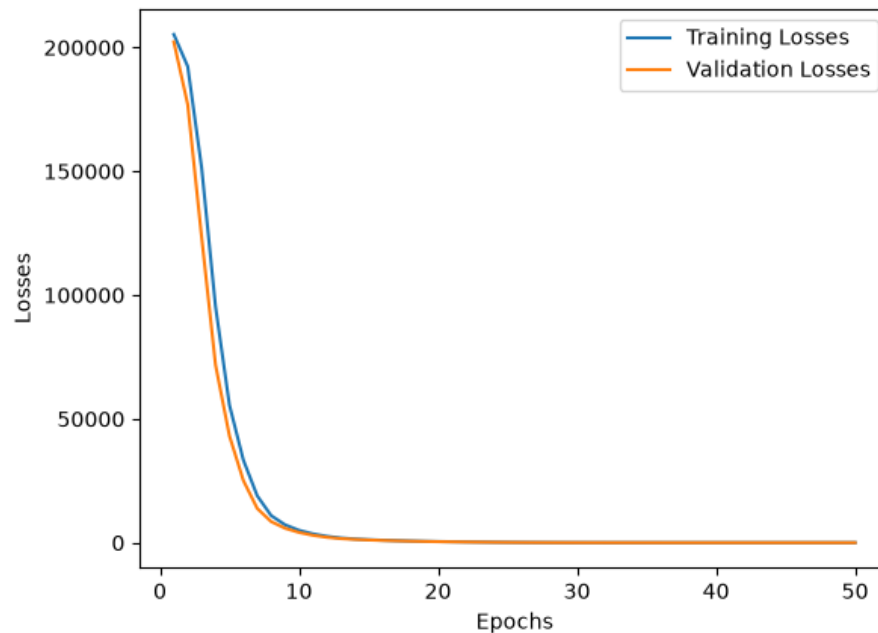


Figure 2: Training loss and validation loss across 50 training epochs.

The curve shows a classic, healthy training pattern: a steep initial drop as the network quickly learns the broad relationship between the sensor readings and energy output, followed by a smooth, gradual flattening as the network fine-tunes its weights. Importantly, the training and validation curves stay close together throughout, with neither one diverging upward — this indicates the model is not overfitting, and is learning a genuinely generalisable relationship rather than memorising the training data.

9. Loading the Best Model and Final Evaluation

Once training was complete, the saved `best_model.pt` checkpoint was reloaded, and the model was evaluated one final time on both the training and test sets, with gradient computation disabled since no further learning was needed at this stage.

Metric	Value
Training MSE	21.4474
Testing MSE	20.3985
Test R ² Score	0.9287

The training and testing MSE values are very close to each other (21.45 vs. 20.40), confirming once again that the model generalises well and is not overfitted to the training data. The **R² score of 0.9287** means the model explains approximately 92.9% of the variance in the plant's actual energy output using just the four ambient sensor readings — a strong result for a real-world physical system, where some amount of unexplained variation is expected due to factors not captured in the dataset.

9.1 Sample Predictions vs. Actual Values

A direct side-by-side comparison of the model's predicted energy output against the actual recorded energy output was generated for the test set. A sample of these predictions is shown below:

Predicted Value (PE)	Actual Value (PE)
435.81	433.27
437.93	438.16
460.93	458.42
475.94	480.82
435.55	441.41
451.84	456.70
432.45	438.04
467.68	467.80
431.97	437.14
456.82	456.78

Across the full test set of 1,914 samples, the predicted values consistently stay very close to the actual recorded energy output, with most predictions differing from the true value by only a few units — consistent with the low MSE and high R^2 score obtained above.

10. Conclusion

This project successfully built, trained, and evaluated a custom Artificial Neural Network in PyTorch to predict a Combined Cycle Power Plant's net energy output from four ambient sensor readings. The complete deep learning workflow was implemented end-to-end — from data preparation and tensor conversion, through defining a multi-layer network architecture, to a proper training loop with validation-based model checkpointing.

The final model achieved a strong test **R^2 score of 0.929** with closely matched training and testing error, showing that a relatively simple two-hidden-layer neural network is well-suited to capturing the relationship between ambient plant conditions and electrical energy output, without overfitting to the training data.